SVKM'S NMIMS
School of Technology Management & Engineering
Department of Computer Engineering

Image and Video Processing

Project Report

SignWords Lab: A Browser-Based, Trainable Sign-Language Word Recogniser with Speech Output

Submitted By:

Harshit Gurjar - 70022400670 (F121)
Priyanshu Agarwal - 70022400718 (F133)
Sourish Bajaj - 70022400583 (F108)
B.Tech CE Section A1

Submitted To:

Dr. Raj Gaurav Mishra Sir
Associate Proffesor

Abstract

Sign-language recognition systems usually depend on fixed vocabularies, large pre-trained models or cloud services, which makes them hard to adapt to a new signer or a new set of words and raises privacy concerns when video leaves the user's device. This project presents SignWords Lab, a web application in which a user trains a sign-language *word* recogniser on their own signs, entirely inside the browser. Frames from a webcam are processed by MediaPipe Hands, which returns 21 landmarks per hand. The landmarks are converted into wrist-relative, scale-normalised feature vectors and stacked into a 24-frame clip that covers 1.2 seconds, sampled every 50 ms, so that hand motion and not only hand shape is learned. A small one-dimensional convolutional network of about 80,000 parameters is trained in TensorFlow.js on the user's own recordings, with data augmentation, live learning curves and a confusion matrix on held-out clips. During detection a sliding window is classified every 150 ms; a word is accepted after three consecutive confident predictions, appended to a sentence and spoken with text-to-speech. A dedicated "No sign (idle)" class lets the system stay silent when the user is not signing. All hand tracking, training and detection run locally and nothing is uploaded. The repository includes an automated smoke test that trains the real model on synthetic data in Node. A formal accuracy evaluation on labelled recordings has not yet been carried out; the measures planned for it are listed in Section 8.

Keywords: sign language, hand landmarks, MediaPipe Hands, temporal sequence classification, 1D CNN, TensorFlow.js, in-browser machine learning, text-to-speech.

1. Introduction

1.1 Background of the Problem

Sign languages are the primary means of communication for many deaf and hard-of-hearing people, yet most hearing people do not know them. Automatic recognition could narrow this gap, but it is difficult: signs are made of hand shape, position and movement over time, signers differ in speed and style, and every sign language has its own vocabulary. Many available systems are trained once on a fixed dataset and offer little room to add words or adapt to a new person. Cloud-based services add a second concern, because they require sending camera video of a person to a remote server.

1.2 Why the Problem Is Relevant

A recogniser that the user can train on their own signs, with a vocabulary they choose, is useful for learning, for demonstrations and for small assistive tools. Running everything in the browser removes installation, removes the need for a server and keeps video on the device. Speaking each recognised word aloud turns the output into something a hearing listener can follow immediately.

1.3 Role of Image and Video Processing

The problem is naturally a video problem. A single frame shows a hand shape, but many words are distinguished by how the hand moves. Image processing finds the hands and their landmarks in each frame, feature extraction turns the landmarks into a compact representation that does not depend on where the hand is in the frame or how large it appears, and temporal processing groups frames into fixed-length clips so that motion becomes learnable. A classifier then maps the clip to a word.

1.4 Motivation for Selecting the Project

The project covers the main stages of a video-processing pipeline (acquisition, detection and landmark extraction, feature construction, temporal modelling, classification and decision logic) in a single working application that can be demonstrated live. It also forces attention to practical questions such as how to avoid false detections when the user is not signing, and how to avoid repeating a word several times for one sign.

2. Problem Statement and Objectives

Problem statement: Given a live webcam stream, let a user define their own vocabulary of sign-language words, record examples, train a recogniser on those examples, and then recognise the words in real time, assembling them into a sentence that is spoken aloud, with all processing done locally in the browser.

Objectives of this project:

1. Capture webcam video and extract 21 hand landmarks per hand with MediaPipe Hands.
2. Convert landmarks into position- and scale-independent features and build fixed-length clips of 24 frames (1.2 s).
3. Let the user record about 30 clips per word, plus an idle class, and store them locally.
4. Train a small 1D convolutional network in the browser with data augmentation, and report loss, accuracy and a confusion matrix on held-out clips.
5. Run a sliding-window detector that accepts a word only after repeated confident predictions, adds it to a sentence and speaks it.
6. Provide a laptop-oriented interface with three workflow tabs (Collect, Train, Detect) and an automated test of the training code.

3. Scope of the Project

What the project covers:

- Live webcam input in Chrome or Edge served from `http://localhost`.
- Hand landmark extraction (up to two hands), feature construction and clip sampling at a fixed rate.
- User-defined vocabulary of isolated words with an additional idle class.
- In-browser training, evaluation, saving and loading of the model; import and export of recorded clips.
- Real-time detection with a decision rule, a running sentence and text-to-speech output with UI chimes.
- A light/dark responsive interface using self-hosted Manrope and Phosphor icons, with no frontend build step.

What the project does not cover:

- Continuous sign-language translation or grammar. The system recognises isolated words and joins them with spaces.
- A ready-made vocabulary or a pre-trained model for any specific national sign language. The user trains their own.
- Facial expression and body pose, which carry meaning in real sign languages. Only hands are used.
- Signer-independent recognition. The model is trained on the person who recorded the clips.
- Mobile browsers, multiple cameras or server-side deployment.

Type of images and videos: RGB webcam video from a fixed laptop camera. **Application domain:** assistive technology, education and human–computer interaction.

4. Dataset / Input Description

Source of input: A live webcam. There is no external dataset; the training data are created by the user inside the application. Each training example is a *clip*: a sequence of 24 feature frames covering 1.2 seconds, sampled every 50 ms.

Dataset name: User-recorded clips (stored locally in the browser's IndexedDB, with import and export of the dataset).

Recommended number of samples: about 30 clips per word and at least 10 idle clips (hands resting or leaving the frame), recorded with small variations in speed, position and distance.

Number of classes: the user's words plus one idle class ("No sign").

Input resolution and format: the webcam frame as delivered by the browser. The classifier does not see pixels; it sees landmark coordinates only.

5. Methodology / Proposed Approach

The system works in three stages: collect, train, detect. The figure below shows the processing pipeline.

```
Webcam
  |
  v
MediaPipe Hands (21 landmarks x up to 2 hands)
  |
  v
Features (wrist-relative, scale-normalised, + wrist position)
  |
  v
Sampler: one feature frame every 50 ms -> 24-frame clip (1.2 s)
  |
  v
Conv1D network (~80k parameters, trained in TensorFlow.js)
  |
  v
Recogniser: window classified every 150 ms; accept after 3
confident predictions in a row
  |
  v
Sentence text (word + space)  +  text-to-speech
```

5.1 Landmark extraction

MediaPipe Hands runs on the webcam stream in the browser and returns 21 landmarks for each detected hand, for up to two hands. The hand model and the libraries are loaded once per session from content delivery networks, so an internet connection is needed at start-up; after that, processing is local.

5.2 Feature construction

Raw landmark coordinates change with where the hand is in the frame and how close it is to the camera. To remove these effects the landmarks are expressed relative to the wrist and normalised by scale. The wrist position is added back as a separate feature so that the model can still use where in the frame a sign is made. Frames are pushed into a rolling buffer.

5.3 Clip sampling

Features are sampled at a fixed rate of one frame every 50 ms. A clip is 24 consecutive samples, which is 1.2 seconds. Using a sequence instead of a single frame allows the model to learn movement as well as hand shape. During recording, the user captures clips for each word; the idle class teaches the model what "not signing" looks like.

5.4 Model and training

A small one-dimensional convolutional network of about 80,000 parameters takes a 24-frame clip and outputs a probability for each class. Convolutions run along the time axis. Training is done in the browser with TensorFlow.js using the recorded clips and data augmentation. The interface shows live loss and accuracy curves, a training log and a confusion matrix computed on held-out clips. The trained model is saved in the browser and is marked outdated if the word list changes.

5.5 Detection and decision rule

In the Detect tab a sliding 1.2-second window is classified every 150 ms. A word is accepted only after three confident predictions in a row. The accepted word is added to the sentence followed by a space and is spoken. The user returns the hands to rest between repeats of the same word, which lets the idle class reset the counter. This is a temporal filter in the same spirit as a debounce: single noisy predictions do not produce output.

Pseudocode of the detection loop:

```
every 50 ms:
    features = extract(landmarks(frame))
    buffer.push(features)           # keep the last 24 frames

every 150 ms:
    probs = model(buffer)           # clip of 1.2 s
    label, conf = argmax(probs)
    if label != idle and conf is high enough:
        streak[label] += 1
    else:
        reset streaks
    if streak[label] == 3:
        sentence += label + ' '
        speak(label)
```

6. Techniques / Algorithms Used

The table lists the image and video processing techniques from the course outline and states whether each is used in SignWords Lab.

Technique	Used?	How it is used or why it is not
Grayscale conversion	No	The hand model works on the colour camera frame; no separate conversion is done by the project.
Histogram equalization	No	No contrast enhancement is applied. The tips advise even lighting and a plain background instead.
Thresholding	Partly	A confidence criterion and a three-in-a-row rule are applied to class probabilities. No pixel-level threshold is applied.
Filtering	No spatial filter	No image filtering by the project. Temporal filtering of predictions is used (Section 5.5).
Edge detection	No	Not needed; hand landmarks come from MediaPipe.
Morphological operations	No	Not used.

Technique	Used?	How it is used or why it is not
DFT / DCT	No	Not used.
Image segmentation	No	No pixel segmentation. The hand detector localises the hand directly.
Feature extraction	Yes	21 landmarks per hand are turned into wrist-relative, scale-normalised features plus wrist position.
Frame extraction	Yes	Frames are taken from the live webcam stream and sampled every 50 ms into clips.
Background subtraction	No	Not used; hands are located by a learned detector.
Optical flow	No	Not computed. Motion is represented by the change of landmark features across the 24 frames.
Motion detection	Implicit	Motion is learned by the network from the clip. The idle class separates signing from rest.
Object detection	Yes	MediaPipe Hands detects hands and their landmarks.
Object tracking	Via MediaPipe	Hand tracking between frames is handled inside MediaPipe Hands. The project adds no separate tracker.
Frame differencing	No	Not used.
Temporal sequence classification	Yes	A 1D CNN classifies each 1.2 s clip.

Hand landmark detection. MediaPipe Hands is a pre-trained model that finds hands in an image and predicts 21 3D landmarks for each. It supplies the geometric features on which the whole system is built.

1D convolution over time. Filters slide along the 24 time steps and learn short movement patterns; deeper layers combine them into whole-sign patterns. Compared with recurrent models, a small convolutional network trains quickly, which matters because training happens in the user's browser.

Data augmentation. Augmentation is applied during training to make the network less sensitive to small changes in speed, position and distance.

Temporal filtering of decisions. Requiring three confident predictions in a row, and an idle class, reduces false triggers and repeated words.

7. Implementation

Programming languages: JavaScript (ES modules), HTML and CSS for the application; Python 3 for the local development server (`serve.py`); Node.js for the optional test.

Software and tools: Chrome or Edge, a code editor, Git and GitHub. No frontend build step is required.

Libraries and frameworks:

- TensorFlow.js (version 4.22 or later, listed as a development dependency) for model building, training and inference.
- MediaPipe Hands for hand landmarks, loaded from a CDN together with the hand model.
- The browser's Web Speech text-to-speech for spoken output, with simple audio chimes in the interface.
- IndexedDB for storing clips locally.

- Manrope (variable font) and Phosphor icons, self-hosted under `assets/` with their licences.

Hardware: a laptop or desktop with a webcam. Inference and training run in the browser, so no server or special hardware is required.

Project structure:

```
signwords-lab/
  index.html        page shell (3 tabs: Collect, Train, Detect)
  serve.py          local dev server
  css/styles.css
  assets/           self-hosted typeface, icons, licences
  js/
    main.js         entry point, shared app state, camera wiring
    config.js       tunables (sequence length, thresholds, keys)
    core/
      camera.js     webcam + MediaPipe hand tracking
      features.js   landmarks -> features, rolling buffer
      sampler.js    fixed-rate sampling + clip capture
      dataset.js    IndexedDB storage, import/export
      model.js      architecture, augmentation, training, eval
      recognizer.js predictions -> 'word detected' events
      speech.js     text-to-speech and chimes
    ui/
      tabs.js dataPanel.js trainPanel.js livePanel.js
      chart.js (learning curves) toast.js
  tests/smoke_train.mjs   trains the real model on synthetic data
```

Important implementation steps:

- Start `python3 serve.py` and open `http://localhost:8000`. The camera and ES modules need localhost, so the page cannot be opened by double-clicking the file.
- In Collect, add words and record clips, including idle clips.
- In Train, train the network and check the curves and the confusion matrix. Record more clips for confused words and retrain.
- In Detect, start the camera. Recognised words are appended to the sentence and spoken.
- Use Export dataset to back up the clips or move them to another computer.
- Run `npm install` and `npm test` to build the model, train it on synthetic signs in Node and check that it learns.

The interface is organised as a navigation rail, a large camera workspace and an open vocabulary list. The three workflow tabs can be operated with the arrow keys, Home and End, and the colour scheme follows the device's light or dark setting.

8. Results and Discussion

8.1 Input versus Output

In the finished application the camera view shows the tracked hand landmarks, the Train tab shows the learning curves and the confusion matrix, and the Detect tab shows the live sentence with each word appended and spoken as it is accepted.

8.2 Automated Smoke Test

The repository contains `tests/smoke_train.mjs`, run with `npm test`. It builds the real model, trains it on synthetic signs in Node and checks that it learns. This test verifies that the model-building and training code runs end to end and that the network can fit separable data. It does not measure how well the system recognises real signs.

8.3 Recognition Accuracy on Recorded Signs

No labelled evaluation results are reported here, because none are available in the repository. The table defines the measures planned for the evaluation, with the status of each.

Measure	Definition	Status
Held-out accuracy	Share of held-out clips classified correctly, from the Train tab	[Fill in]
Per-word precision and recall	From the confusion matrix, one row per word and for the idle class	[Fill in]
False-trigger rate	Words added to the sentence per minute while the user is not signing	Not yet measured
Repeat/miss rate	Share of deliberate signs that were missed or counted twice	Not yet measured
Latency	Time from the end of a sign to the word being spoken	Not yet measured
Effect of clip count	Held-out accuracy with 10, 20 and 30 clips per word	Not yet measured
Effect of conditions	Accuracy under different lighting, backgrounds and distances	Not yet measured

8.4 Discussion

The design choices have clear consequences. Using wrist-relative, scale-normalised landmarks removes dependence on where the signer sits and how large the hand appears, so a small number of clips per word can be enough. Using a 1.2-second clip lets the model separate words that share a hand shape but differ in movement. The idle class and the three-in-a-row rule trade a little speed for fewer false words. The README advises 30 or more clips per word and states that words differing only slightly in motion need more clips, which is consistent with the behaviour expected from a small model trained on a small personal dataset.

9. Challenges / Limitations

Small personal datasets. Because the user records their own clips, the model is trained on a small number of examples from one person. Accuracy is expected to drop for a different signer, camera position or lighting.

Isolated words only. The system recognises single words. It does not model sign-language grammar, and it does not use facial expression or body pose, which carry meaning in real signing.

Hand tracking quality. Recognition depends on MediaPipe finding the hands. Fast movement, motion blur, hands overlapping each other or leaving the frame, poor lighting and cluttered backgrounds can lead to missing or wrong landmarks. The README recommends even lighting and a plain background.

Similar signs. Words that differ only slightly in motion are easily confused and need more clips. The confusion matrix is the tool for finding such pairs.

Vocabulary changes require retraining. Changing the word list makes the saved model outdated.

Decision rule trade-off. Waiting for three confident predictions adds delay and requires the user to return the hands to rest between repeated words, which is unnatural for fluent signing.

Evaluation not yet done. No labelled test set or measured accuracy, latency or false-trigger rate is available. The smoke test uses synthetic data and therefore shows only that the code can learn, not how it performs on real signs.

Browser and network dependence. The app needs Chrome or Edge on localhost, and needs an internet connection once per session to load TensorFlow.js, MediaPipe and the hand model from CDNs. Training time depends on the user's computer.

Privacy. Hand tracking, training and detection run locally and nothing is uploaded. Clips are kept in the browser's storage and leave the device only if the user exports them. This is an advantage for an application that uses a camera.

10. Conclusion

What was implemented. A complete in-browser pipeline for training and using a custom sign-language word recogniser: webcam capture, hand landmark extraction, feature construction, clip sampling, local dataset storage, a trainable 1D CNN with evaluation tools, a sliding-window detector with a decision rule, and speech output, together with an interface and an automated smoke test.

Techniques used. MediaPipe hand landmark detection and tracking, geometric feature normalisation, fixed-rate temporal sampling, 1D convolutional sequence classification, data augmentation and temporal filtering of predictions.

What was achieved. The application lets a user go from an empty vocabulary to a speaking recogniser without leaving the browser, and keeps all video on the device.

Future work. Collect labelled test recordings from several signers and report accuracy, precision, recall, latency and false-trigger rate; add face and pose features; try signer-independent training; support continuous signing instead of isolated words; and compare the 1D CNN with recurrent and transformer models.

References

1. Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools.
2. Gonzalez, R. C., & Woods, R. E. (2018). Digital Image Processing (4th ed.). Pearson.
3. Lugaresi, C., et al. (2019). MediaPipe: A framework for building perception pipelines. arXiv:1906.08172.
4. Zhang, F., et al. (2020). MediaPipe Hands: On-device real-time hand tracking. arXiv:2006.10214.
5. Smilkov, D., et al. (2019). TensorFlow.js: Machine learning for the web and beyond. Proceedings of Machine Learning and Systems (SysML).
6. Bai, S., Kolter, J. Z., & Koltun, V. (2018). An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. arXiv:1803.01271.
7. SignWords Lab source code and README. https://github.com/Agarwalrishu13/signwords-lab